

Subject Name:  
**PROGRAMMING IN JAVA**

Subject Code :  
**BCAC 0023**

# Course Objective

- **Objective:**
- The objective of this course is that the students will understand the foundation of good programming skills by discussing key issues to the design of object-oriented programming.

# Course Outcome

- **Outcome:** *After completion of course, student will be able to:*
- CO1: Understand differences between procedures oriented and object oriented approach.
- CO2: Understand the relevance of Object Oriented Programming techniques.
- CO3: Understand how to write, compile and execute a Java Program.
- CO4: Understand the use of polymorphism and Inheritance.
- CO5: Understand how to define user exceptions and its uses.
- CO6: Understand what is a thread and Multithreading model.
- CO7: Understand how to develop a GUI application.

# BCAC 0023: PROGRAMMING IN JAVA

**Educators –**

**Dr. Vinod Jain**

**Mr. Sachin Sharma**

**Mr. Mayank Agrawal**

**Dr. Aditya Saxena**

**Mr. DP Yadav**

**Mr. Shivank Chauhan**

# Syllabus

- Module 1 – 15 Lectures
- Module 2 – 15 Lectures

# BCAC 0023: PROGRAMMING IN JAVA

**Objective:** *The objective of this course is that the students will understand the foundation of good programming skills by discussing key issues to the design of object-oriented programming.*

**Credits: 02**

**L-T-P: 2-0-0**

| Module No. | Content                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Teaching Hours |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| I          | <p><b>Introduction:</b> Introduction to Java, History of <u>Java</u>, Features of Java, Importance of Java, Byte Code, JVM, JRE, JDK, JIT, Java Implementation, Application of Java, Sample Program &amp; Compilation, Lexical Issues (White Space, Identifiers, Literals, Comments, Separators, Keyword).</p> <p><b>Data Type, Operators, Control Structures:</b> Variables, Constants, Declaration, Literals, Scope of Variable, Type Casting, Arithmetic Operators, Relational Operators, Logical Operators, Assignment Operator, Dot Operators, Control Statements.</p> <p><b>Array:</b> Declaration, Creation, Initialization, Length, Two-Dimensional Arrays, Command-Line Arguments.</p> <p><b>String Handling:</b> Predefined Functions in String, String Methods</p> <p><b>Introduction of Classes, Objects and Methods:</b> Introduction to Object Class Defining Class, Adding Variables, Adding Methods, Creating Objects,</p> <p><b>Constructors:</b> Types of Constructors, Keyword this &amp; static, Garbage Collection</p> | 15             |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| II | <p><b>Inheritance:</b> Inheritance, Types of Inheritance, Creating Multilevel Hierarchy, Method Overloading &amp; Overriding, Dynamic Method Dispatching, final keyword, Abstract Class.</p> <p><b>Packages And Interfaces:</b> Defining Interfaces. Extending and Implementing Interfaces</p> <p>Defining Packages, Access Protection, Importing Packages,</p> <p><b>Exception Handling:</b> Exception Types, Multiple Catch Clauses, Nested Try Statements, Throw, Throws, Finally, Java's Built-in Exceptions, Creating Your Own Exception Subclasses.</p> <p><b>Multithreaded Programming:</b> Thread Life Cycle, Creating Threads, Thread Methods, Thread Priority</p> <p><b>Introduction to Applet Programming-</b> Introduction to Applet, Applet Architecture, Applet Life Cycle, Applet Class, Applet Tag, Passing Parameters, Use of Graphics Class, Applet Methods.</p> | 15 |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|

# Books

- **Text Book:**

- Naughton, Schildt, *“The Complete Reference JAVA2”*, 7th edition., TMH.

- **Reference Books:**

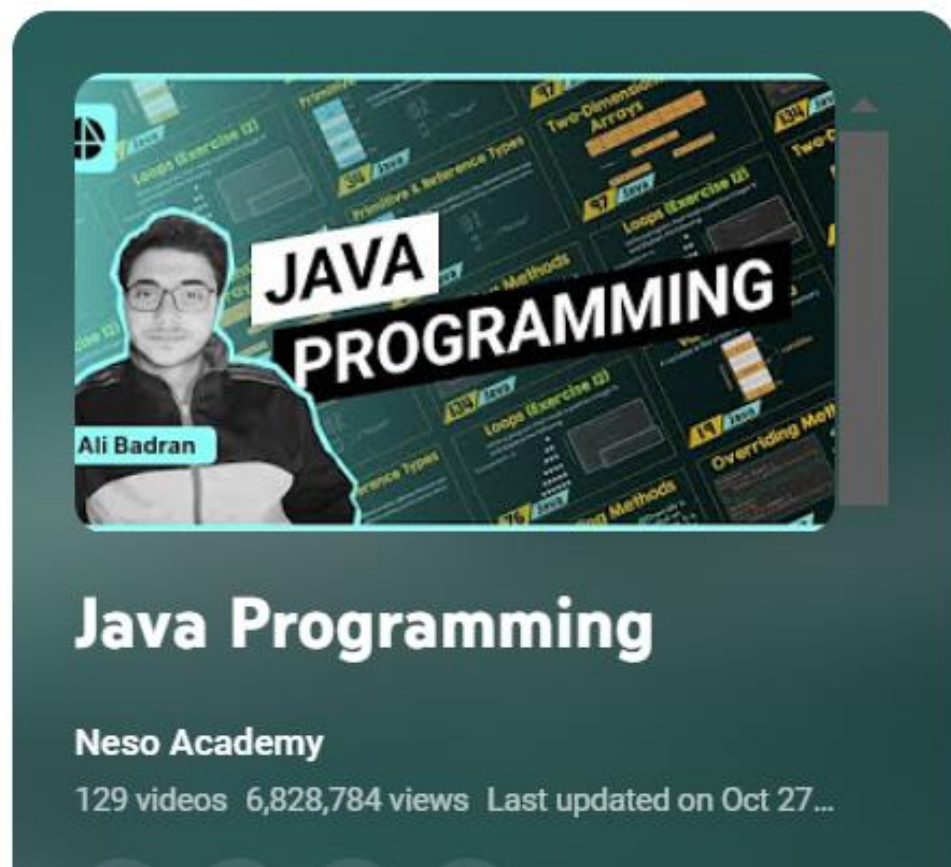
- Naughton Patric, Morrisison Michel, (2006), *“Java Hand Book”*, Osborne/McGraw-Hill, 5th edition.
- **Balaguruswami E., (2007), *“Programming with Java”*, TMH, 3rd edition.**
- Decker & Hirshfied, (2000), *“Programming Java”*, Vikas Publication 2nd edition.
-

# Online Resources

- <https://www.javatpoint.com/java-tutorial>
- <https://www.w3schools.com/java/>
- <https://www.geeksforgeeks.org/java/>
- <https://www.tutorialspoint.com/java/index.htm>

# Online Resources

- Youtube Tutorials
- [https://youtube.com/playlist?list=PLBlnK6fEyqRjKA\\_NuK9mHmlk0dZzuP1P5&si=PKfRH4PyjcpuycAl](https://youtube.com/playlist?list=PLBlnK6fEyqRjKA_NuK9mHmlk0dZzuP1P5&si=PKfRH4PyjcpuycAl)
- [https://youtube.com/playlist?list=PLmRclvVt5DtnqhXTJwd-oqVRwO3bLZCGV&si=rPbSD\\_PSsNIXneEV](https://youtube.com/playlist?list=PLmRclvVt5DtnqhXTJwd-oqVRwO3bLZCGV&si=rPbSD_PSsNIXneEV)



[https://youtube.com/playlist?list=PLBlnK6fEyqRjKA\\_NuK9mHmlk0dZzuP1P5&si=PKfRH4PyjcpuycAl](https://youtube.com/playlist?list=PLBlnK6fEyqRjKA_NuK9mHmlk0dZzuP1P5&si=PKfRH4PyjcpuycAl)

[https://youtube.com/playlist?list=PLmRclvVt5DtnqhXTJwd-oqVRwO3bLZCGV&si=rPbSD\\_PSsNIXneEV](https://youtube.com/playlist?list=PLmRclvVt5DtnqhXTJwd-oqVRwO3bLZCGV&si=rPbSD_PSsNIXneEV)

# History of Java

- The history of Java is very interesting.
- The history of Java starts with the Green Team. Java team members (also known as Green Team), initiated this project to develop a language for digital devices such as set-top boxes, televisions, etc. However, it was best suited for internet programming.
- Java was developed by James Gosling, who is known as the father of Java, in 1995.
- James Gosling and his team members started the project in the early '90s.

# History of Java

- The principles for creating Java programming were "Simple, Robust, Portable, Platform-independent, Secured, High Performance, Multithreaded, Architecture Neutral, Object-Oriented, Interpreted, and Dynamic".
- Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc. Following are given significant points that describe the history of Java.

# History of Java

- 1) James Gosling, Mike Sheridan, and Patrick Naughton initiated the Java language project in June 1991. The small team of sun engineers called Green Team.
- 2) Initially it was designed for small, embedded systems in electronic appliances like set-top boxes.
- 3) Firstly, it was called "Greentalk" by James Gosling, and the file extension was .gt.
- 4) After that, it was called Oak and was developed as a part of the Green project.
- Why Java was named as "Oak"?
- Java History from Oak to Java

# History of Java

- 5) Why Oak? Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.



# History of Java

- 6) In 1995, Oak was renamed as "Java" because it was already a trademark by Oak Technologies.
- Why Java Programming named "Java"?
- 7) Why had they chose the name Java for Java language? The team gathered to choose a new name. The suggested words were "dynamic", "revolutionary", "Silk", "jolt", "DNA", etc.
- **Java was finalized by the team.** James Gosling tabled Java title based on type of espresso coffee bean.
- Java is an island in Indonesia where new coffee was discovered termed as Java coffee. As per James Gosling, Java was among the top choice along with Silk. Finally Java was selected as it was quite unique and represented the essence of being dynamic,revolutionary and fun to say.

# History of Java

- |      |                   |             |      |            |             |
|------|-------------------|-------------|------|------------|-------------|
| • 1  | JDK Beta          | 1995        | • 13 | JAVA SE 11 | 5 Sep 2018  |
| • 2  | JDK 1.0           | 23 Jan 1996 | • 14 | JAVA SE 12 | 19 Mar 2019 |
| • 3  | JDK 1.1           | 19 Feb 1997 | • 15 | JAVA SE 13 | 17 Sep 2019 |
| • 4  | JDK 1.2           | 8 Dec 1998  | • 16 | JAVA SE 14 | 17 Mar 2020 |
| • 5  | JDK 1.3           | 8 May 2000  | • 17 | JAVA SE 15 | 15 Sep 2020 |
| • 6  | JDK 1.4           | 6 Feb 2002  | • 18 | JAVA SE 16 | 16 Mar 2021 |
| • 7  | JDK 1.5 or J2SE 5 | 30 Sep 2004 | • 19 | JAVA SE 17 | 14 Sep 2021 |
| • 8  | JAVA SE 6         | 11 Dec 2006 | • 20 | JAVA SE 18 | 22 Mar 2022 |
| • 9  | JAVA SE 7         | 7 Jul 2011  | • 21 | JAVA SE 19 | 20 Sep 2022 |
| • 10 | JAVA SE 8         | 18 Mar 2014 | • 22 | JAVA SE 20 | 21 Mar 2023 |
| • 11 | JAVA SE 9         | 21 Sep 2017 | • 22 | JAVA SE 21 | 19 Sep 2023 |
| • 12 | JAVA SE 10        | 20 Mar 2018 |      |            |             |

# Introduction to Java

- Java is a popular programming language, created in 1995.
- It is owned by Oracle, and more than **3 billion** devices run Java.
- It is used for:
  1. Mobile applications (specially Android apps)
  2. Desktop applications
  3. Web applications
  4. Web servers and application servers
  5. Games
  6. Database connection
  7. And much, much more!

# Importance of Java: Why Use Java?

1. Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
2. It is one of the most popular programming language in the world
3. It is easy to learn and simple to use
4. It is open-source and free
5. It is secure, fast and powerful
6. It has a huge community support (tens of millions of developers)
7. Java is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs
8. As Java is close to C++ and C#, it makes it easy for programmers to switch to Java or vice versa

# Features of Java : Java buzzwords

- The features of Java are also known as java *buzzwords*.
- Important features of java are

1. Simple
2. Object-Oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Interpreted
9. High Performance
10. Multithreaded
11. Distributed
12. Dynamic

# Simple

- Java is very easy to learn, and its syntax is simple, clean and easy to understand.
- Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.

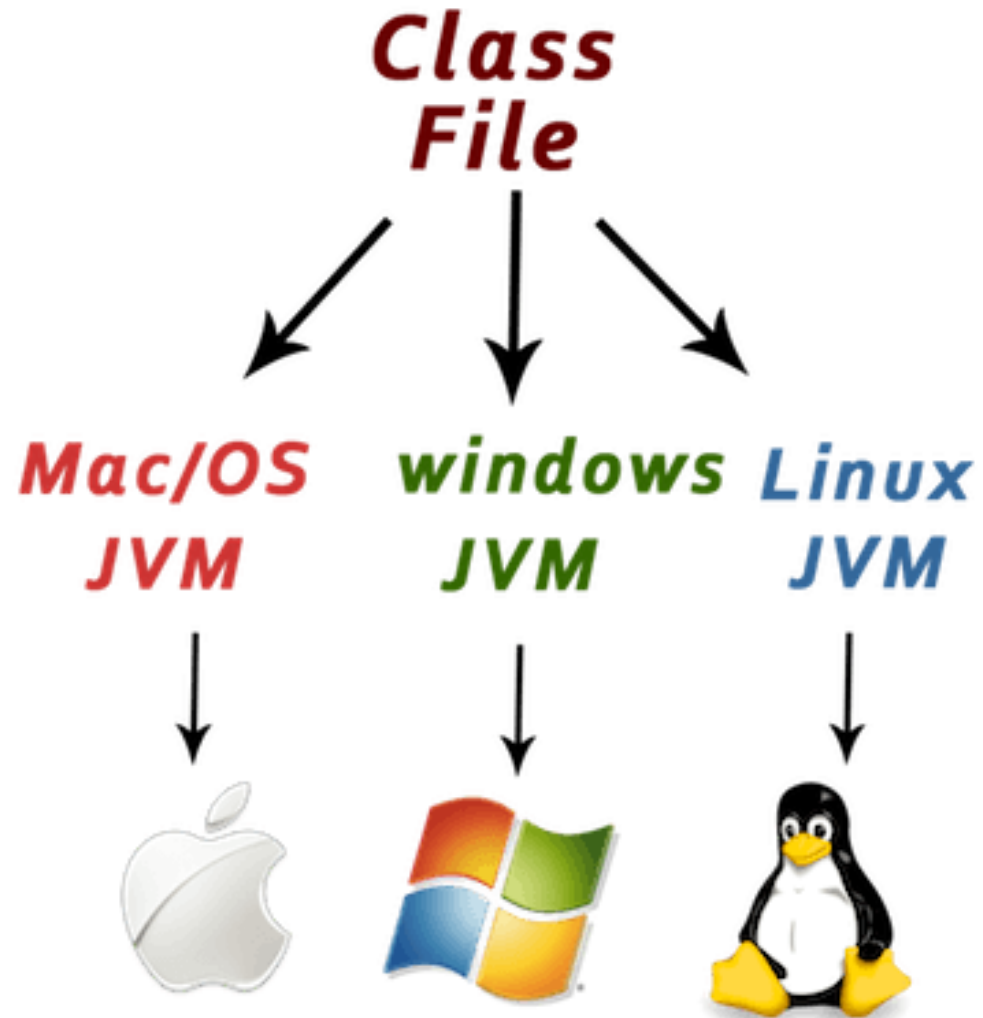
# Object-Oriented

- Java is an [object-oriented](#) programming language.
- Basic concepts of OOPs are:
  1. Object
  2. Class
  3. Inheritance
  4. Polymorphism
  5. Abstraction
  6. Encapsulation

# Portable

- Java is portable because it facilitates you to carry the Java bytecode to any platform.
- It doesn't require any implementation.

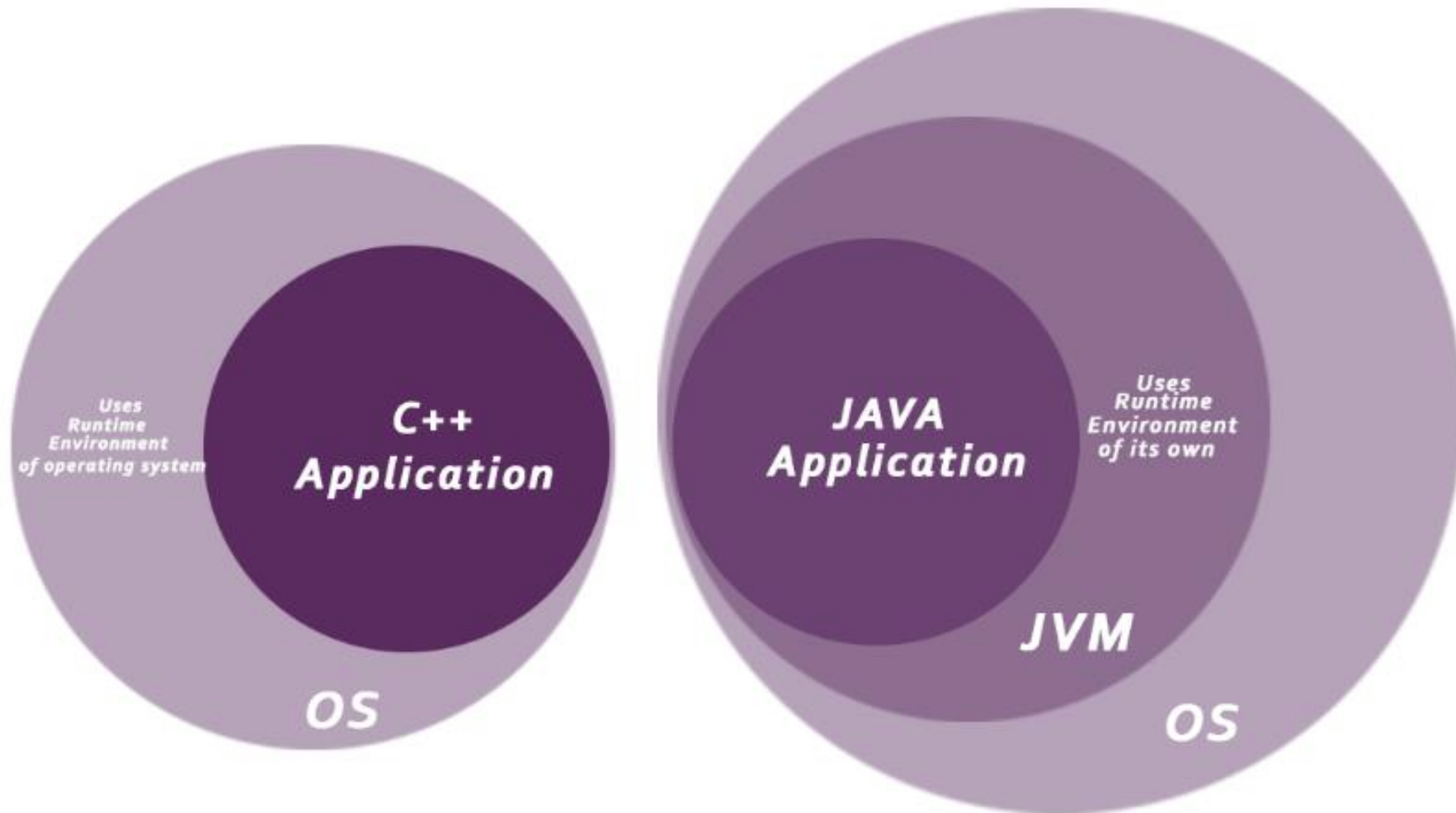
# Platform Independent



# Platform Independent

- Java is a write once, run anywhere language.
- A platform is the hardware or software environment in which a program runs.
- Java programs can be run on any platform.
- Java code can be run on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode.
- This bytecode is a platform-independent code because it can be run on multiple platforms.

# Secured



# Secured

- Java is best known for its security.
- With Java, we can develop virus-free systems.
- Java is secured because:
- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**
- **Classloader**
- **Bytecode Verifier**
- **Security Manager**

Robust

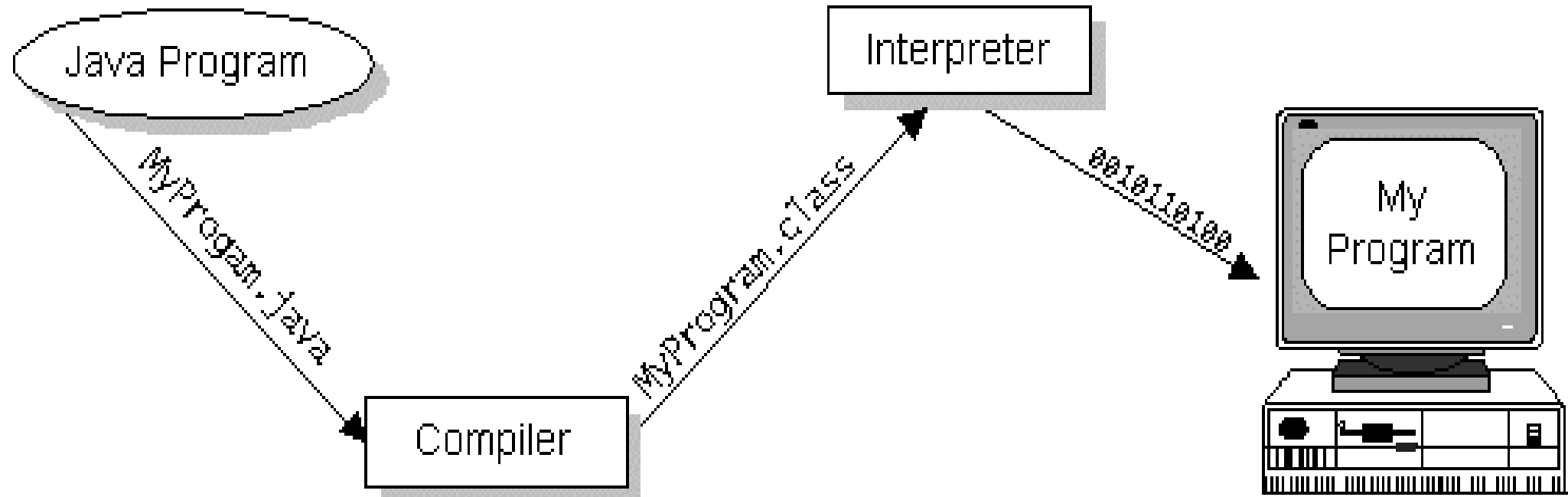
# Robust

- Robust simply means strong.
- Java is robust because:
  1. It uses strong memory management.
  2. There is a lack of pointers.
  3. There is automatic garbage collection.
  4. There are exception handling and the type checking mechanism in Java.
  5. All these points make Java robust.

# Architecture-neutral

- Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.
- In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture.
- However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

# Interpreted



# Interpreted

- Java is both compiled and interpreted.
- Java byte code is translated on the fly to native machine instructions and is not stored anywhere.
- Java code is compiled first and then interpreted.
- Most of the features of java depends on bytecode eg. Security, Portable, platform independent etc.

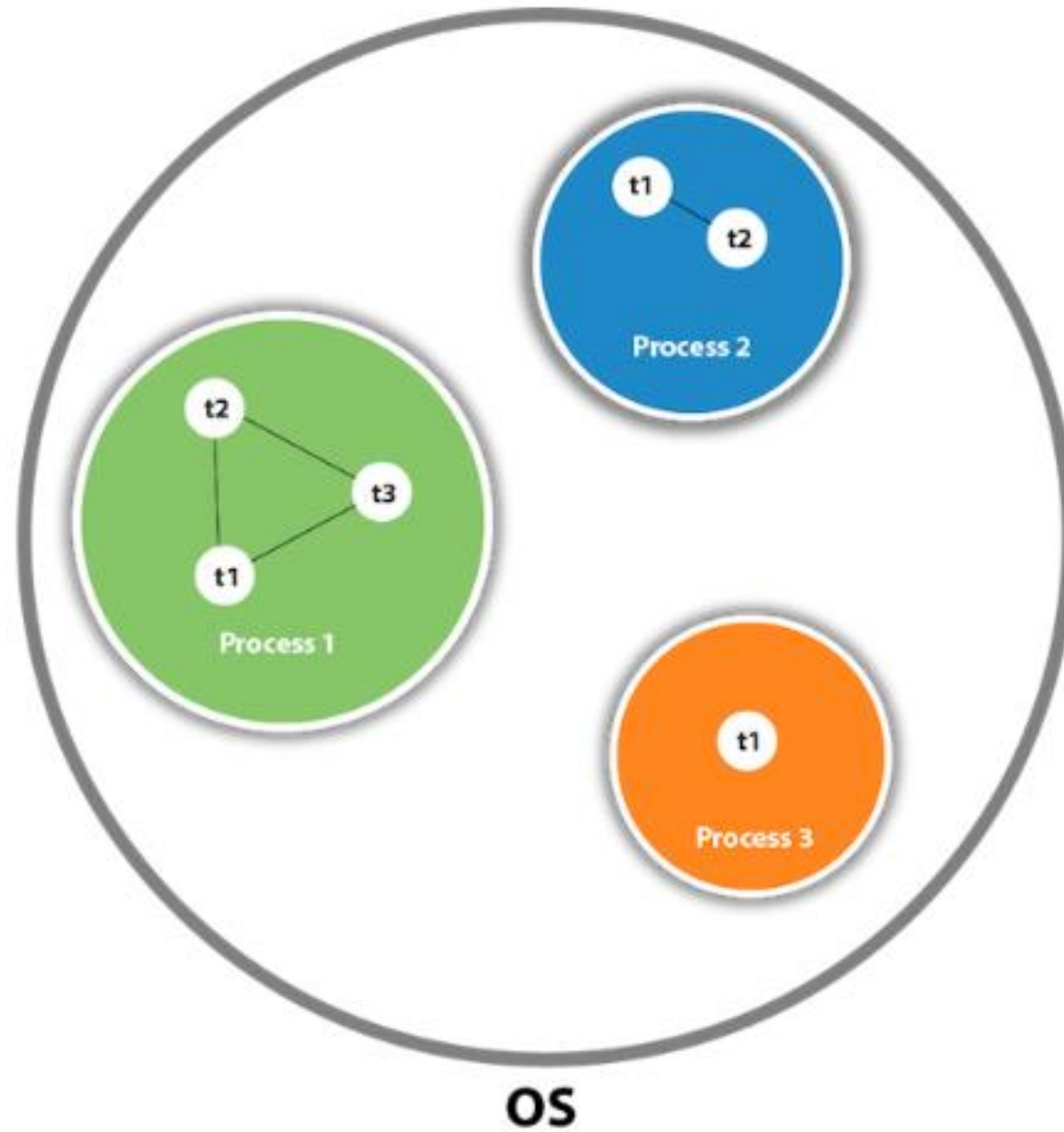
# High-performance

- Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code.
- It is still a little bit slower than a compiled language (e.g., C++).
- Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

# Distributed

- Java is distributed because it facilitates users to create distributed applications in Java.
- RMI (Remote Method Invocation) and EJB (Enterprise java beans) are used for creating distributed applications.

# Multi-threaded



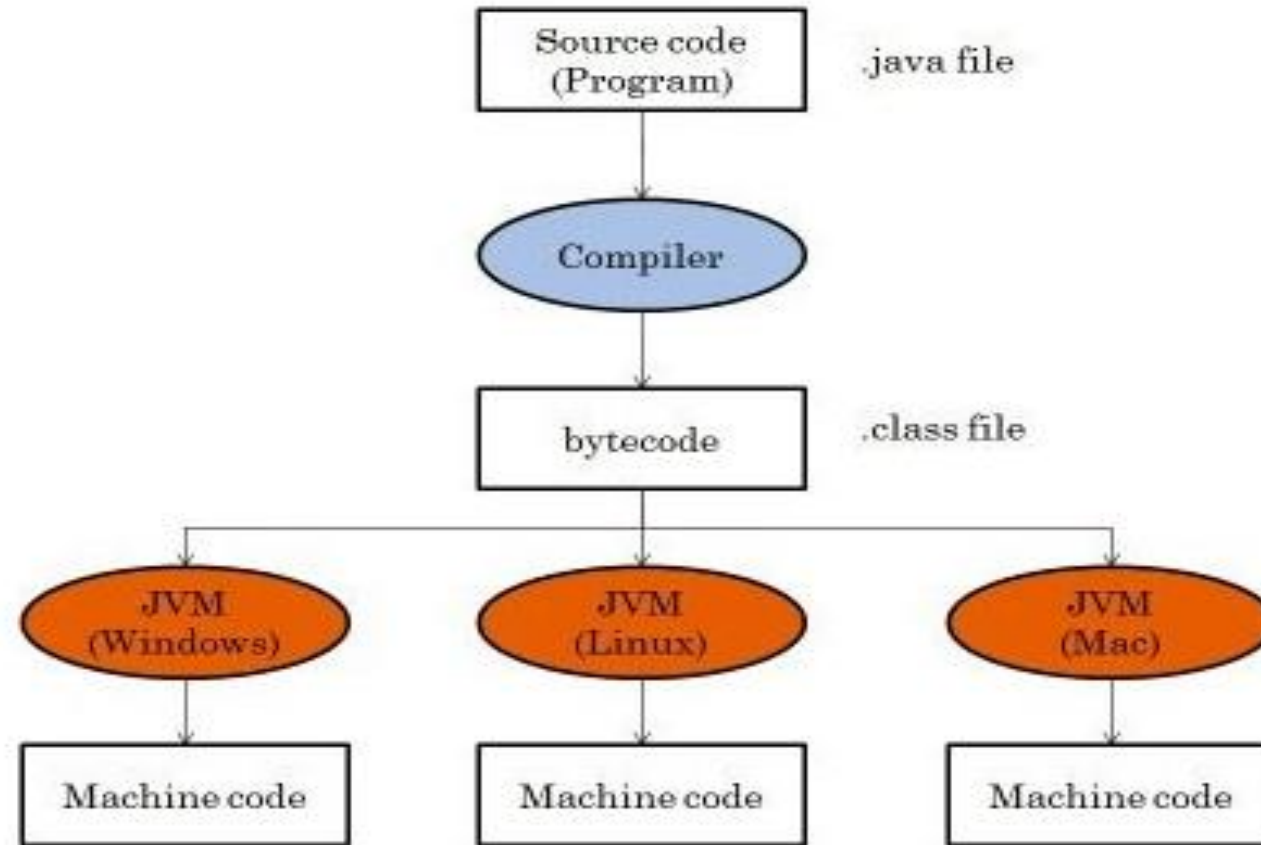
# Multi-threaded

- A thread is like a separate program, executing concurrently.
- We can write Java programs that deal with many tasks at once by defining multiple threads.
- The main advantage of multi-threading is that it doesn't occupy memory for each thread.
- It shares a common memory area.

# Dynamic

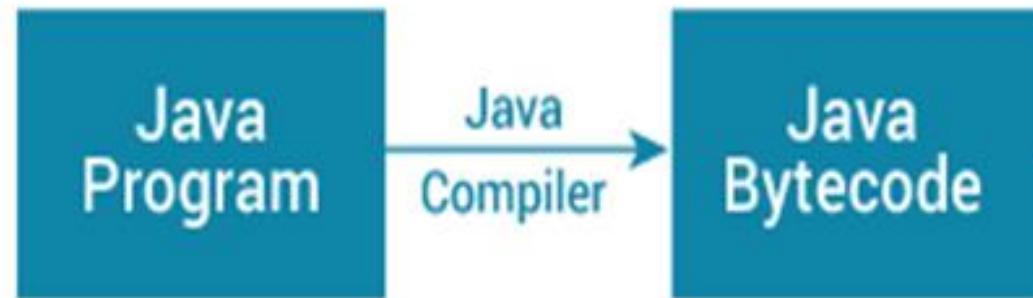
- Java is a dynamic language.
- It supports dynamic loading of classes. It means classes are loaded on demand.

# Bytecode



# Bytecode

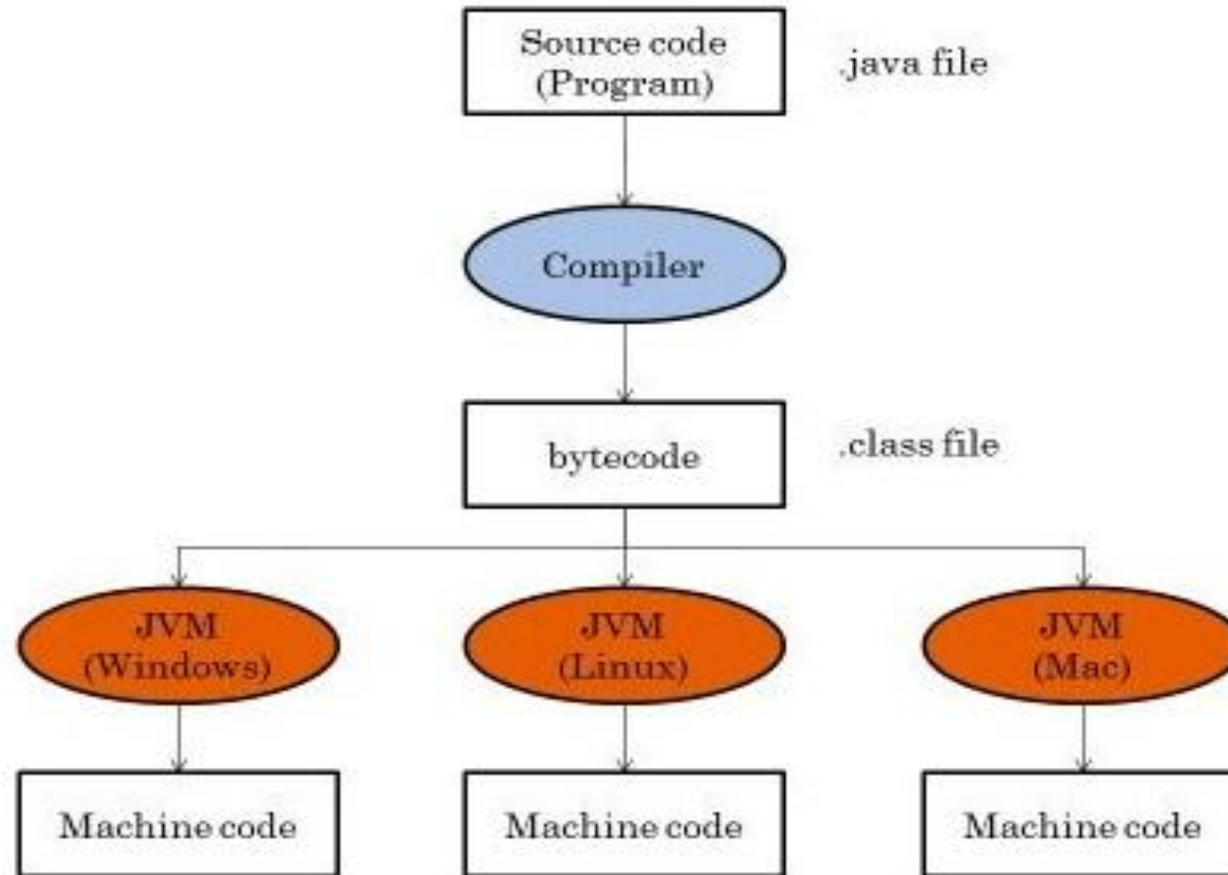
- As soon as a java program is compiled, java bytecode is generated.
- Java bytecode is the instruction set for the Java Virtual Machine.
- Java bytecode is the machine code in the form of a .class file.
- With the help of java bytecode we achieve platform independence in java.



# Advantage of Java Bytecode

- Platform independence - Bytecode which helps us to achieve this.
- Java is a platform-independent language.
- It's because when you write Java code, it's ultimately written for JVM but not your physical machine (computer).
- Since, JVM executes the Java bytecode which is platform independent, Java is platform-independent.

# JVM Java Virtual Machine



# JVM Java Virtual Machine

- JVM (Java Virtual Machine) is an abstract machine that enables your computer to run a Java program.
- When you run the Java program, Java compiler first compiles your Java code to bytecode.
- **Then, the JVM translates bytecode into native machine code**  
(set of instructions that a computer's CPU executes directly).



# How Does the JVM Work?

- The JVM is divided into three main subsystems:
  - ClassLoader Subsystem – For loading classes
  - Runtime Data Area – For storing objects and variables
  - Execution Engine – Contains Interpreter and JIT compiler and Garbage collector
- 
- Ref
  - <https://dzone.com/articles/jvm-architecture-explained>

# What is JRE?

- JRE (Java Runtime Environment) is a software package that provides Java class libraries, along with Java Virtual Machine (JVM), and other components to run applications written in Java programming.
- JRE is the superset of JVM.



# What is JDK?

JDK

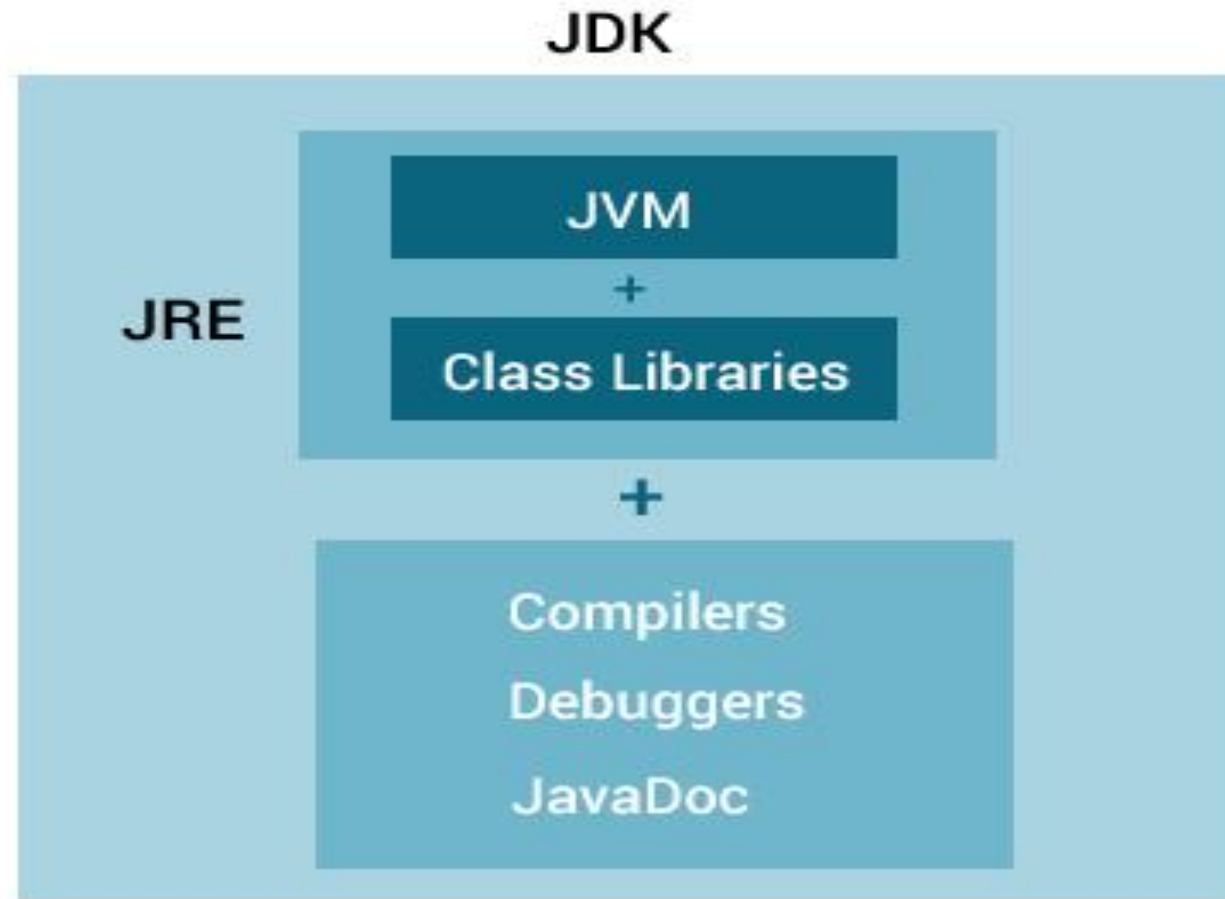
JRE

+ Compilers + Debuggers ...

# What is JDK?

- JDK (Java Development Kit) is a software development kit to develop applications in Java.
- When you download JDK, JRE is also downloaded, and don't need to download it separately.
- In addition to JRE, JDK also contains number of development tools (compilers, JavaDoc, Java Debugger etc).

Here's the relationship between JVM, JRE, and JDK.



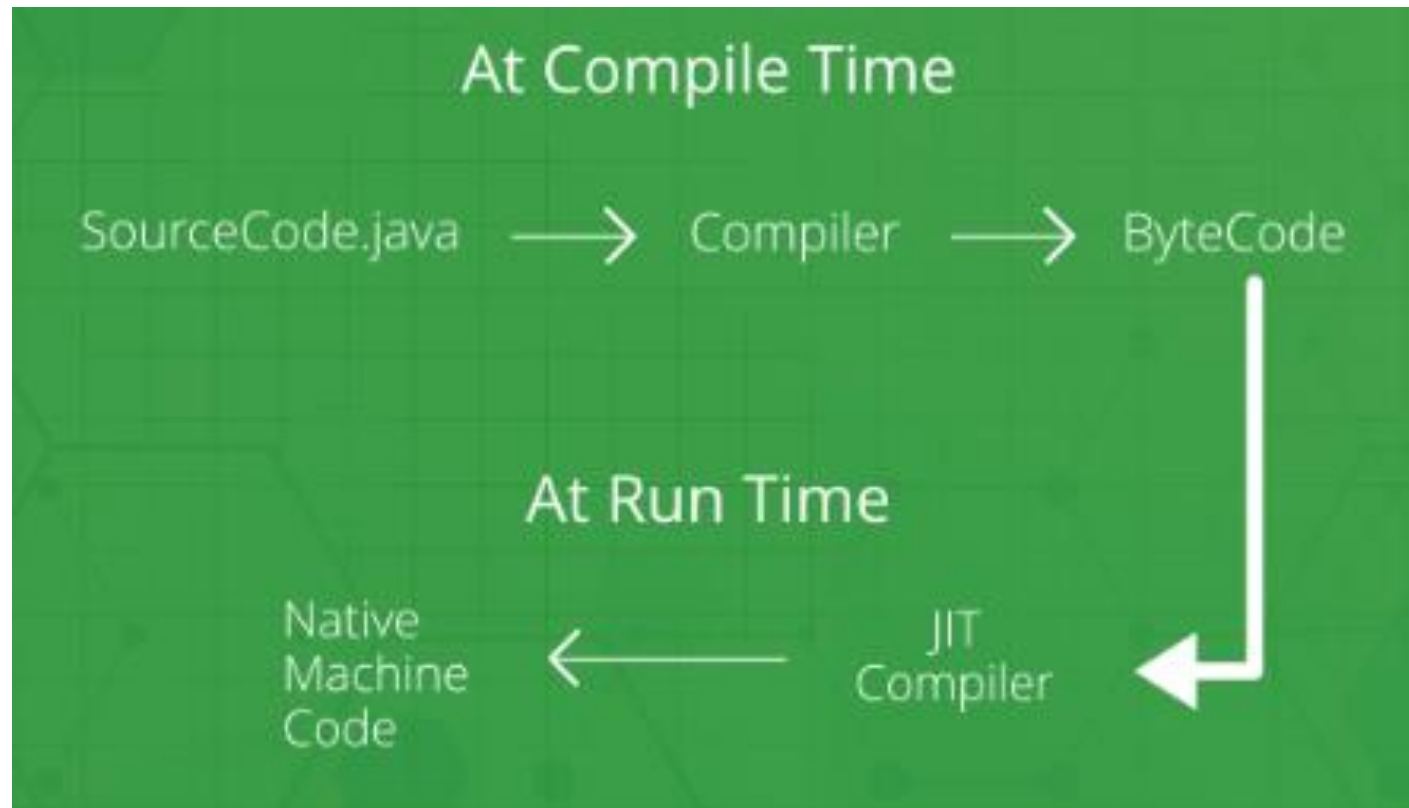
# JIT

- The Bytecode have to be interpreted or compiled to proper machine instructions depending on the instruction set architecture.
- Interpreting the bytecode affects the speed of execution.
- In order to improve performance, JIT compilers interact with the Java Virtual Machine (JVM).

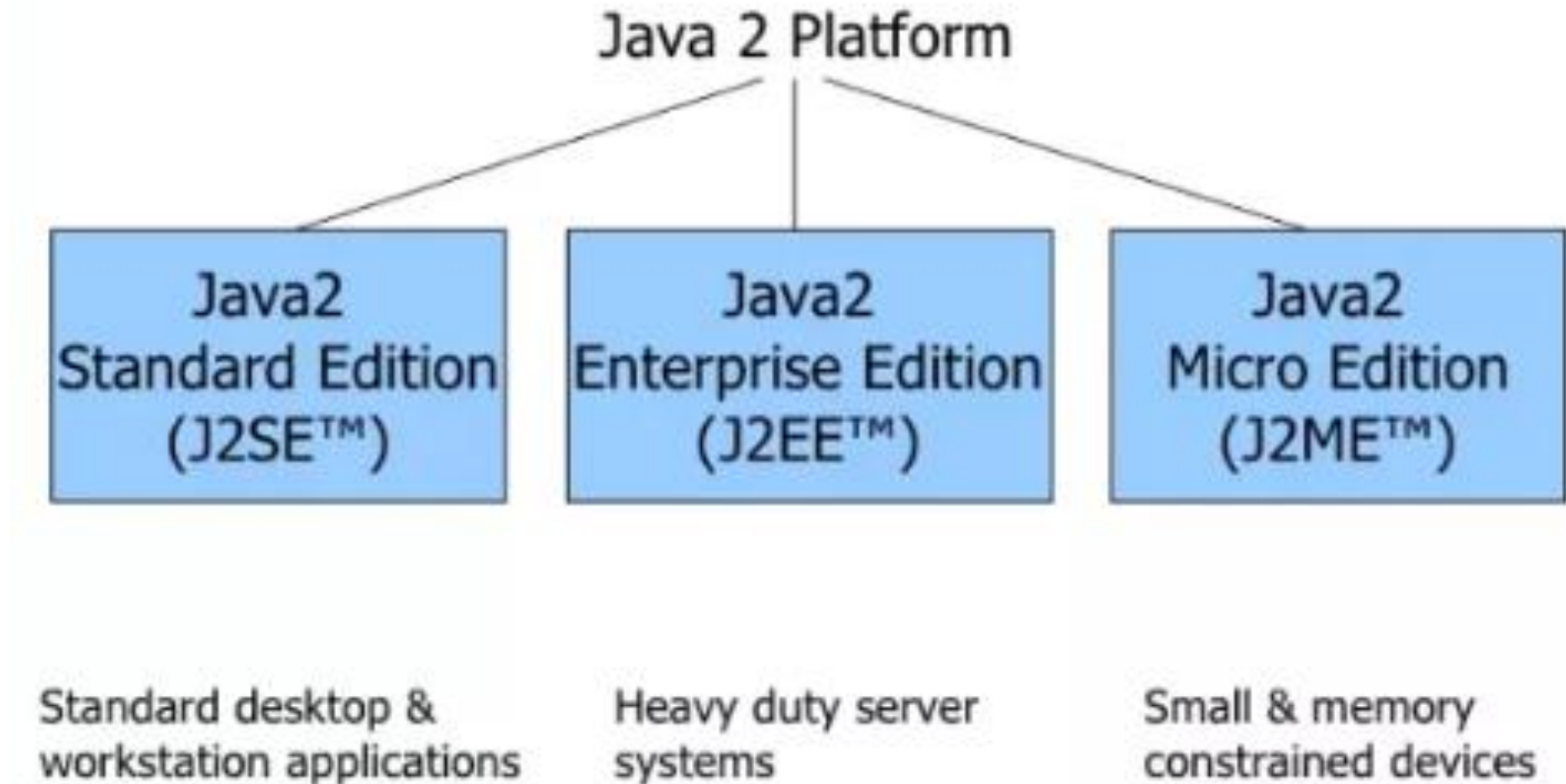
# JIT

- The Just-In-Time (JIT) compiler is an essential part of the JRE that is responsible for performance optimization of java based applications at run time.
- In order to improve performance, JIT compilers interact with the Java Virtual Machine (JVM) at run time and compile suitable bytecode sequences into native machine code.
- The JIT compiler is able to perform certain simple optimizations while compiling a series of bytecode to native machine language.

# JIT



# J2SE,J2EE,J2ME



# J2SE(Java Platform, Standard Edition)

- This is the most basic and standard version of Java.
- It's the purest form of Java, a basic foundation for all other editions.
- It consists of a wide variety of general purpose API's (like java.lang, java.util) as well as many special purpose APIs.

# J2SE(Java Platform, Standard Edition)

- J2SE is mainly used to create applications for **Desktop environment**.
- It consist all the basics of Java the language, **variables, primitive data types, Arrays, Streams, Strings Java Database Connectivity(JDBC)** and much more.
- This is the **standard, from which all other editions came out**, according to the needs of the time.

# J2ME(Java Platform, Micro Edition)

- This version of Java is mainly concentrated for the applications running on embedded systems, mobiles and small devices.(which was a constraint before it's development)
- The basic aim of this edition was to work on mobiles, wireless devices, set top boxes etc.
- The J2ME (Micro Edition) runs on portable, mobile, handheld devices.
- Most of the apps, developed for the phones (prior to smartphones era), were built on J2ME platform only(the .jar apps on Nokia app store).

# J2EE(Java Platform, Enterprise Edition)

- The Enterprise version of Java has a much larger usage of Java, like development of web services, networking, server side scripting and other various web based applications.
- J2EE is a community driven edition, i.e. there is a lot of continuous contributions from industry experts, Java developers and other open source organizations.

# J2EE(Java Platform, Enterprise Edition)

- J2EE uses many components of J2SE, as well as, has many new features of it's own like Servlets, JavaBeans, Java Message Services, adding a whole new functionalities to the language.
- J2EE uses HTML, CSS, JavaScript etc., so as to create web pages and web services.
- Sun's J2SE package is less than 35 MB, while the J2EE is 98 MB.
- <https://www.geeksforgeeks.org/j2se-vs-j2me-vs-j2ee-whats-the-difference/>
- <https://wiki.lunarpages.com/J2ME - J2SE - J2EE>

# J2EE(Java Platform, Enterprise Edition)

- References
- <https://www.geeksforgeeks.org/j2se-vs-j2me-vs-j2ee-whats-the-difference/>
- <https://wiki.lunarpages.com/J2ME - J2SE - J2EE>

# Java Interview Questions

- **Question 1: What are the main differences between J2SE, J2EE, and J2ME?**

# Java Interview Questions

- J2SE (Java 2 Standard Edition):
- Purpose: It is used for developing general-purpose standalone applications. It provides core functionalities such as the Java language basics, collections, threading, I/O, networking, AWT, and Swing for GUI development.
- Key Components: Core Java libraries, JVM, development tools (like javac, jar), etc.
- Target: Desktop applications.

# Java Interview Questions

- J2EE (Java 2 Enterprise Edition):
- Purpose: It is used for developing large-scale, distributed, and component-based enterprise applications. It includes APIs and runtime environments for scripting, messaging, web services, etc.
- Key Components: Servlets, JSP, EJB, JMS, JPA, JDBC, etc.
- Target: Web and enterprise-level applications.

# Java Interview Questions

- J2ME (Java 2 Micro Edition):
- Purpose: It is used for developing applications for small devices like mobile phones, PDAs, embedded systems, etc. It provides a subset of J2SE APIs with additional libraries specific to small devices.
- Key Components: CLDC (Connected Limited Device Configuration), MIDP (Mobile Information Device Profile), etc.
- Target: Mobile and embedded applications.

# Sample Program & Compilation

// WAP to print all command line arguments

```
class cm3
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        int i;
```

```
        String str = "Java";
```

```
        for(i=0;i<str.length();i++)
```

```
        {
```

```
            System.out.print("\n " + str.charAt(i));
```

```
        }
```

```
    }
```

```
}
```

# Lexical Issues

- Java programs are a collection of whitespace, identifiers, literals, comments, operators, separators, and keywords.
- These are called lexical issues.
- The operators are described in the next section.
- The others are described here.

# White Space

- White space is a character that is invisible.
- In Java, whitespace is a space, tab, or newline.
- White spaces are used to separate tokens
- Eg.
- `int x = 10;`
- Here a space between `int` and `x` is a whitespace that separate `int`(keyword) and `x`(identifier).

# Identifiers

- Identifiers are used for class names, method names, and variable names.

# Find Identifiers in this program

```
class cm3
{
    public static void main(String args[])
    {
        int i;
        String str = "Java";
        for(i=0;i<str.length();i++)
        {
            System.out.print("\n " + str.charAt(i));
        }
    }
}
```

```
// WAP to print all command line arguments
class cm3
{
    public static void main(String args[])
    {
        int i;
        String str = "Java";
        for(i=0;i<str.length();i++)
        {
            System.out.print("\n " + str.charAt(i));
        }
    }
}
```

# Literals

- A constant value in Java is called *literal*.
- *Literals may be of type integer, float, string, character etc.*
- *Eg. 10, 3.14, 100, "Hello Java ", 'Y' etc*

# Comments

- Comments are lines in the java code which are used to explain the code i.e. its purpose, logic etc.
- It makes the source code self explanatory
- In Java Comments are of three types
  1. Single line comment by //
  2. Multi line comment by /\* and \*/
  3. *Documentation comment* used to produce an HTML file that documents your program. The documentation comment begins with a /\*\* and ends with a \*/.

# Separators

- In Java, there are a few characters that are used as separators.
- The most commonly used separator in Java is the semicolon, it is used to terminate statements.

# Separators

| Symbol | Name        | Purpose                                                                                                                                                                                              |
|--------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ( )    | Parentheses | Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types. |
| { }    | Braces      | Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.                                                         |
| [ ]    | Brackets    | Used to declare array types. Also used when dereferencing array values.                                                                                                                              |
| ;      | Semicolon   | Terminates statements.                                                                                                                                                                               |
| ,      | Comma       | Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a <b>for</b> statement.                                                                   |
| .      | Period      | Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.                                                                   |

# The Java Keywords

- There are some words whose meaning is predefined in java and These keywords cannot be used as names for a variable, class, or method.
- Eg. For, if, else, void, int etc.

# TABLE of Java Keywords

## **Keywords In Java**

|              |                |               |                  |
|--------------|----------------|---------------|------------------|
| 1. abstract  | 13. double     | 25. int       | 37. strictfp     |
| 2. assert    | 14. else       | 26. interface | 38. super        |
| 3. boolean   | 15. enum       | 27. long      | 39. switch       |
| 4. break     | 16. extends    | 28. native    | 40. synchronized |
| 5. byte      | 17. final      | 29. new       | 41. this         |
| 6. case      | 18. finally    | 30. package   | 42. throw        |
| 7. catch     | 19. float      | 31. private   | 43. throws       |
| 8. char      | 20. for        | 32. protected | 44. transient    |
| 9. class     | 21. if         | 33. public    | 45. try          |
| 10. continue | 22. implements | 34. return    | 46. void         |
| 11. default  | 23. import     | 35. short     | 47. volatile     |
| 12. do       | 24. instanceof | 36. static    | 48. while        |

# References

- <https://www.javatpoint.com/features-of-java>
- Complete Reference
- <https://www.tutorialspoint.com/What-are-the-major-features-of-Java-programming>

# References

- <https://www.javatpoint.com/java-tutorial>
- <https://www.w3schools.com/java/>
- <https://www.geeksforgeeks.org/java/>
- <https://www.tutorialspoint.com/java/index.htm>